

Robot Learning: From Fundamentals to Foundation Models

Lecture 5: Reinforcement Learning II

Oier Mees

ETH zürich



Microsoft

16.03.2026

Recap: RL Algorithm Taxonomy & Trade-offs

Exact Methods	Tabular RL	Deep RL (Discrete)	Deep RL (Continuous)
Value Iter. Policy Iter. Q-Val. Iter.	Q-Learning SARSA	DQN Double DQN	DDPG QT-Opt (CEM)

Needs Dynamics Model

✓ Train+Test

✗ Model-free

✗ Model-free

✗ Model-free

State Space

Discrete, small

Discrete, small

High-dim images

High-dim images

Action Space

Discrete

Discrete

Discrete

Continuous

Policy (train time)

Tabular

Tabular

Neural Net

Actor + Critic NNs

Off-policy

N/A

Q-Learn ✓
SARSA ✗

✓ (Replay Buffer)

✓ (Replay Buffer)

Convergence

✓ Guaranteed

✓ Tabular+explore

✗ No guarantee

✗ Brittle

Key Innovation

Bellman
contraction

TD updates from
experience

Replay buffer +
Target network

Actor learns
arg max_a Q

Value-Based Methods: learn what every possible actions is worth & hope argmax will extract the best policy

Value-Based Methods: learn what every possible actions is worth & hope argmax will extract the best policy

Policy Gradients: what if we optimize our policy directly to make good actions more likely?

Recap: The Learning Objective

- Accumulated reward an agent receives:

$$G = \sum_{t \geq 0} \gamma^t r_t$$

- Expected Return:

$$J(\pi) = \mathbb{E}_{\tau \sim p_{\pi}(\tau)} \left[\sum_{t \geq 0} \gamma^t r_t \right]$$

- Optimal Policy:

$$\pi^* = \arg \max_{\pi} J(\pi)$$



Recap: Trajectory Probability

- Probability of a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ under policy π :

- $$p_{\pi}(\tau) = \underbrace{\rho(s_0)}_{\text{Initial State Distribution}} \prod_{t=0}^{T-1} \underbrace{\pi_{\theta}(a_t|s_t)}_{\text{Policy}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{Transition Function}}$$

Initial State
Distribution

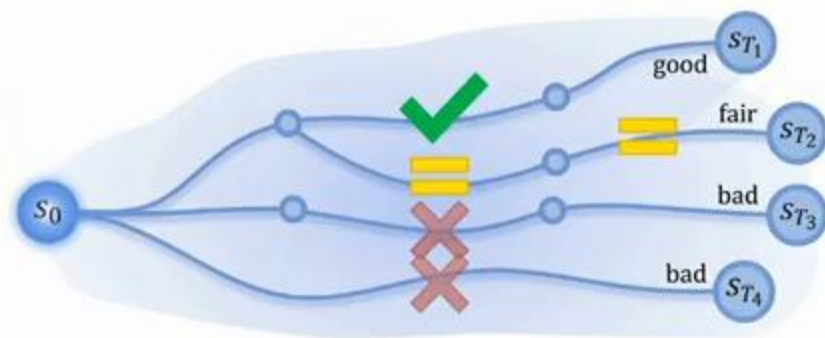
Policy

Transition Function



How Good is the Policy?

- Let $\pi_{\theta}(a|s)$ be the policy we optimize
- $\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\sum_{t=0}^{T-1} R(s_t, a_t)]}_{J(\theta)}$



- $J(\theta) = \mathbb{E}_{\tau \sim p_{\pi}(\tau)} [R(\tau)] \approx \frac{1}{N} \sum_i R(\tau_i) = \frac{1}{N} \sum_i \sum_{t=0}^{T-1} r_{i,t}$

Monte Carlo Sampling: Sum over samples from π_{θ}

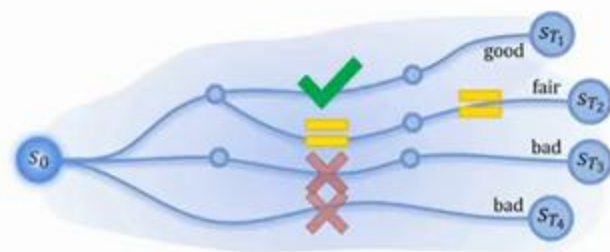
How to get the Gradient?

- Let $\pi_{\theta}(a|s)$ be the policy we optimize
- $\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\sum_{t=0}^{T-1} R(s_t, a_t)]}_{J(\theta)}$

- $R(\tau) = \sum_{t=0}^{T-1} R(s_t, a_t)$

- $J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)]$

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) R(\tau) d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau\end{aligned}$$



How to get the Gradient?

Chain Rule

$$\nabla_{\theta} f(\mathbf{x}(\theta)) = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \nabla_{\theta} x_i(\theta)$$

$$\nabla_{\theta} \log(p_{\theta}(\tau)) = \frac{1}{p_{\theta}(\tau)} \nabla_{\theta} p_{\theta}(\tau)$$

Policy Gradient Theorem:

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau$$

Can't sample this

How to get the Gradient?

Chain Rule

$$\nabla_{\theta} f(\mathbf{x}(\theta)) = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \nabla_{\theta} x_i(\theta)$$

$$\nabla_{\theta} \log(p_{\theta}(\tau)) = \frac{1}{p_{\theta}(\tau)} \nabla_{\theta} p_{\theta}(\tau)$$

$$\underbrace{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \nabla_{\theta} p_{\theta}(\tau)$$

Log Derivative Trick $\nabla \log x = \frac{\nabla x}{x}$

Policy Gradient Theorem:

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau$$

Can't sample this

$$= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau$$

$$= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

We can sample this!

Sampling still non-differentiable, but we can differentiate wrt to θ !

How to get the Gradient?

Trajectory Distribution $p_{\theta}(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t|s_t) P(s_{t+1}|s_t, a_t)$

$$\log p_{\theta}(\tau) = \log p(s_0) + \sum_{t=1}^T \log \pi_{\theta}(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)$$

Policy Gradient Theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

$$\nabla_{\theta} [\cancel{\log p(s_0)} + \sum_{t=1}^T \log \pi_{\theta}(a_t|s_t) + \cancel{\log p(s_{t+1}|s_t, a_t)}]$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right) \left(\sum_{t=0}^{T-1} R(s_t, a_t) \right) \right]$$

REINFORCE

1. Initialize π_θ
2. While not converged do
 - I. Collect samples $\{\tau^i\}$ by rolling out policy π_θ
 - II. Compute gradient $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=0}^{N-1} \left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=0}^{T-1} R(s_{i,t}, a_{i,t}) \right) \right]$
 - III. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
3. Return π_θ

Simple statistical gradient-following algorithms for connectionist reinforcement learning

Williams, R. J. (1992)

Intuition Behind Gradient

- Policy Gradient:


Imitation gradient, weighted by reward →  Upweight good trajectories
 Downweight bad ones

- BC Gradient:

$$\nabla_{\theta} J_{BC}(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \right]$$

Intuition Behind Gradient

- Policy Gradient:

$$\nabla_{\theta} J_{PG}(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} \left[\underbrace{\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)}_{\text{Imitation gradient, weighted by reward}} \left(\sum_{t=0}^{T-1} R(s_{i,t}, a_{i,t}) \right) \right]$$

Imitation gradient, weighted by reward $\rightarrow$  Upweight good trajectories
 Downweight bad ones

- BC Gradient:

$$\nabla_{\theta} J_{BC}(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \right]$$

Causality

arm moves correctly toward cup, but overshoots.
What will REINFORCE do?



Causality: Policy behavior at t' does not affect reward at time t when $t < t'$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underbrace{\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'})}_{\text{Reward-To-Go}} \right) \right]$$

sum of future rewards

Reward-To-Go

Causality

arm moves correctly toward cup, but overshoots.
What will REINFORCE do?



✓ first half of the trajectory was good!
but REINFORCE **penalizes all the actions** in the trajectory

Causality: Policy behavior at t' does not affect reward at time t when $t < t'$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underbrace{\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'})}_{\text{Reward-To-Go}} \right) \right]$$

sum of future rewards

Reward-To-Go

High Variance

τ^1 : arm moves vaguely toward cup but overshoots



$R = 99$

τ^2 : arm reaches cup perfectly



$R = 101$

REINFORCE: Both have great return, lets make both behaviors more likely!

$$\Delta\theta \approx \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot R(\tau)$$

High Variance

τ^1 : arm moves vaguely toward cup but overshoots



$R = 99$

τ^2 : arm reaches cup perfectly



$R = 101$

REINFORCE: Both have great return, lets make both behaviors more likely!

$$\Delta\theta \approx \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot R(\tau)$$

$$\Delta\theta \approx \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot 99 \quad \leftarrow \text{Huge gradient for your optimizer!}$$

Effective signal is $101 - 99 = 2$ $\rightarrow$ Signal-to-Noise:
 $2/100 = 2\%$ of the gradient is informative

Reducing Variances with Baselines

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'}) - b \right) \right]$$

subtract a constant baseline, often mean reward



$$R = 99 - 100 = -1$$

$$R = 101 - 100 = 1$$

$$b = \frac{1}{N} \sum_{i=0}^{N-1} R(\tau_i)$$

by subtracting average reward, we get negative gradients for below average behavior ✓

✓ Subtracting a baseline does not change the gradient in expectation, it is unbiased.

Why can we do this?

$$\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau = \int \nabla_{\theta} p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0$$

Advantage Function

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underbrace{\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'})}_{\text{Reward-To-Go}} - b \right) \right]$$

Reward-To-Go

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'}) \mid s_t, a_t \right]$$

What could be the ideal baseline?

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [Q^{\pi}(s, a)]$$



Accounts for the specific difficulty of each state, ensuring the robot is only rewarded for actions that outperform its own average expectation in that situation

Advantage Function

How much better it is to take a action a in state s ?

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t) \right]$$

$Q^\pi(s_t, a_t) = \mathbb{E}_{\pi_\theta} \left[\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'}) \mid s_t, a_t \right]$ - expected return by taking action a from state s

$V^\pi(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot | s)} [Q^\pi(s, a)]$ - expected return from state s

Distribution Shift & Sample Efficiency

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\sum_{t'=t}^{T-1} R(s_{t'}, a_{t'}) - b \right) \right]$$

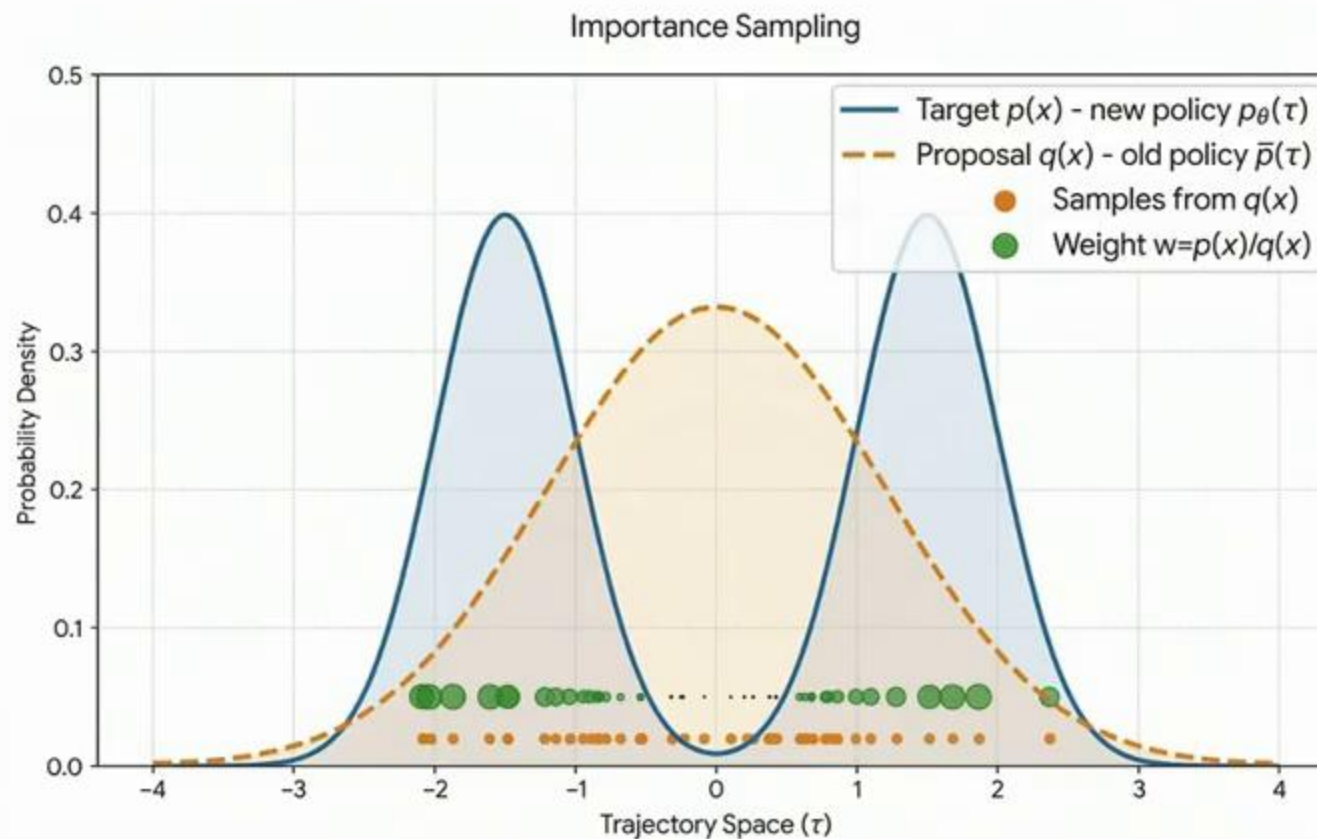
assumes samples from current policy

- I. Collect samples $\{\tau^i\}$ by rolling out policy π_{θ}
- II. Compute gradient $\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^{N-1} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left(\sum_{t=0}^{T-1} R(s_{i,t}, a_{i,t}) \right) \right]$
- III. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$  We need to **throw away old data & collect new data every gradient step!** 🤖

Vanilla policy gradient is On-Policy, unlike Off-Policy algorithms like DQN that can reuse past data

Policy Gradients Off-Policy Version

- Can we use data from previous policy $\bar{p}(\tau)$?



$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x) f(x) dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) dx \\ &= E_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \end{aligned}$$

q needs to have **non-zero support** where $p(x) > 0$ to remain unbiased

Policy Gradients Off-Policy Version

RL objective with importance sampling

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

Policy Gradients Off-Policy Version

RL objective with importance sampling

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

$$\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(s_1)} \prod_{t=1}^T \pi_{\theta}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}}{\cancel{p(s_1)} \prod_{t=1}^T \bar{\pi}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}} = \prod_{t=1}^T \frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\bar{\pi}(\mathbf{a}_t | \mathbf{s}_t)}$$

Policy Gradients Off-Policy Version

RL objective with importance sampling

$$J(\theta) = E_{\tau \sim \bar{p}(\tau)} \left[\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$
$$\frac{p_{\theta}(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(s_1)} \prod_{t=1}^T \pi_{\theta}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}}{\cancel{p(s_1)} \prod_{t=1}^T \bar{\pi}(a_t | s_t) \cancel{p(s_{t+1} | s_t, a_t)}} = \prod_{t=1}^T \frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\bar{\pi}(\mathbf{a}_t | \mathbf{s}_t)}$$

How do derive the gradient to update our new policy $\pi_{\theta'}$ with samples from old π_{θ} ?

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim \mathbf{p}_{\theta'}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) - b \right) \right]$$


new policy

Policy Gradients Off-Policy Version

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) - b \right) \right]$$

↑
↑
 old policy importance weights

Policy Gradients Off-Policy Version

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) - b \right) \right]$$

↑
↑
 old policy importance weights

$$= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \right) \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) - b \right) \right]$$

↑

substitute from previous slide.
 this product can lead to **exploding/vanishing gradients** for large T!

Policy Gradients Off-Policy Version

In practice, we often consider expectations over timesteps instead of trajectories

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_{\theta}(s_{i,t}, a_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$



more stable to compute, but hard to measure

$$\frac{\pi_{\theta'}(s, a)}{\pi_{\theta}(s, a)} = \frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)} \cdot \frac{\pi_{\theta'}(s)}{\pi_{\theta}(s)} \approx \frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)}$$



Often approximate with 1

Final form:

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$

Off-Policy Policy Gradients Algorithm

Take **multiple gradient steps** on the same batch

- 
- I. Collect samples $\{\tau^i\}$ by rolling out policy π_θ
 - II. Compute gradient using $\{\tau^i\}$ samples $\nabla_{\theta'} J(\theta') \approx$
$$\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t}|s_{i,t})}{\pi_\theta(a_{i,t}|s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t}|s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$
 - III. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Entropy Regularization

- What if the support set is zero, $p(x) > 0$ but $q(x) = 0$?
 - New policy $\pi_{\theta'}$ assigns probability to action-state pairs that π_{θ} never took.
 - ✗ Importance weight explodes, estimator gets biased!

Entropy Regularization

- What if the support set is zero, $p(x) > 0$ but $q(x) = 0$?
 - New policy $\pi_{\theta'}$ assigns probability to action-state pairs that π_{θ} never took.
 - ✗ Importance weight explodes, estimator gets biased!
- **Solution:** Add **entropy bonus** to avoid policy becoming too deterministic

Entropy Regularization

- What if the support set is zero, $p(x) > 0$ but $q(x) = 0$?
 - New policy $\pi_{\theta'}$ assigns probability to action-state pairs that π_{θ} never took.
 - ✗ Importance weight explodes, estimator gets biased!
- **Solution: Add entropy bonus** to avoid policy becoming too deterministic

↙ Temperature parameter

$$L(\theta') = L_{PG}(\theta') + \beta \mathbb{H}(\pi_{\theta'}(\cdot | s))$$

Discrete Shannon Entropy

$$\mathbb{H}(\pi_{\theta'}) = - \sum_{a \in A} \pi_{\theta'}(a | s) \log \pi_{\theta'}(a | s)$$

Continuous Differential Entropy

$$\mathbb{H}(\pi_{\theta'}) = \frac{d}{2} (1 + \log(2\pi)) + \sum_{j=1}^d \log \sigma_j$$

Policy Gradients with Constraints

- What if our policy changes a lot before sampling new data?

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$

Unlikely action under π_{θ} gets high reward, $\pi_{\theta'}$ makes it much more likely and gradient explodes! $w = \frac{0.10 \text{ (new)}}{0.005 \text{ (old)}} = 20$

$$\mathbb{E}_{s \sim \pi_{\theta}} [D_{KL}(\pi_{\theta}(\cdot | s) || \pi_{\theta'}(\cdot | s))] \leq \epsilon$$

Policy Gradients with Constraints

- What if our policy changes a lot before sampling new data?

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right) - b \right)$$

Unlikely action under π_{θ} gets high reward, $\pi_{\theta'}$ makes it much more likely and gradient explodes! $w = \frac{0.10 \text{ (new)}}{0.005 \text{ (old)}} = 20$

- Solution:** constrain the policy to not change too much during gradient updates

$$\mathbb{E}_{s \sim \pi_{\theta}} [D_{KL}(\pi_{\theta}(\cdot | s) || \pi_{\theta'}(\cdot | s))] \leq \epsilon$$

Trust Region Policy Optimization (TRPO)

- Maximize advantage from old policy while not deviating too much

$$\text{maximize}_{\theta'} \quad L(\theta') = E_{s,a \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} A^{\pi_{\theta}}(s, a) \right]$$

$$\text{subject to } \mathbb{E}_{s \sim \pi_{\theta}} [D_{KL}(\pi_{\theta}(\cdot | s) || \pi_{\theta'}(\cdot | s))] \leq \epsilon$$

Trust Region Policy Optimization (TRPO)

- Maximize advantage from old policy while not deviating too much

$$\text{maximize}_{\theta'} \quad L(\theta') = E_{s,a \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} A^{\pi_{\theta}}(s, a) \right]$$

$$\text{subject to } \mathbb{E}_{s \sim \pi_{\theta}} [D_{KL}(\pi_{\theta}(\cdot | s) || \pi_{\theta'}(\cdot | s))] \leq \epsilon$$



Monotonic improvement guarantee



Requires the Fisher Information Matrix to solve the constraint optimization, is $O(N^3)$ number of trainable parameters



Conjugate gradient approximation reduces this to $O(kn) \approx O(n)$, but still $\sim 20\times$ more expensive than a standard gradient step



Scaling to deep networks requires additional approximations like K-FAC (Kronecker-Factored Approximate Curvature)

Trust region policy optimization

Schulman, Levine, et al., (2015)

Case Study: Quadruped Locomotion with TRPO

DARPA Subterranean Challenge Urban Circuit



Learning Quadrupedal Locomotion over Challenging Terrain

Lee, Hwangbo, et al., (2020)

Proximal Policy Optimization (PPO)

- What if just clip importance ratio instead of expensive constraints to avoid getting too large?

$$L(\theta') = E_{s,a \sim \pi_\theta} \left[\min \left(\underbrace{\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s,a), \text{clip} \left(\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_\theta}(s,a)} \right) \right]$$

Force the importance sampling ratio to stay within a safe interval

Proximal Policy Optimization (PPO)

- What if just clip importance ratio instead of expensive constraints to avoid getting too large?

$$L(\theta') = E_{s,a \sim \pi_\theta} \left[\min \left(\underbrace{\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s,a), \text{clip} \left(\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_\theta}(s,a)} \right) \right]$$

Force the importance sampling ratio to stay within a safe interval

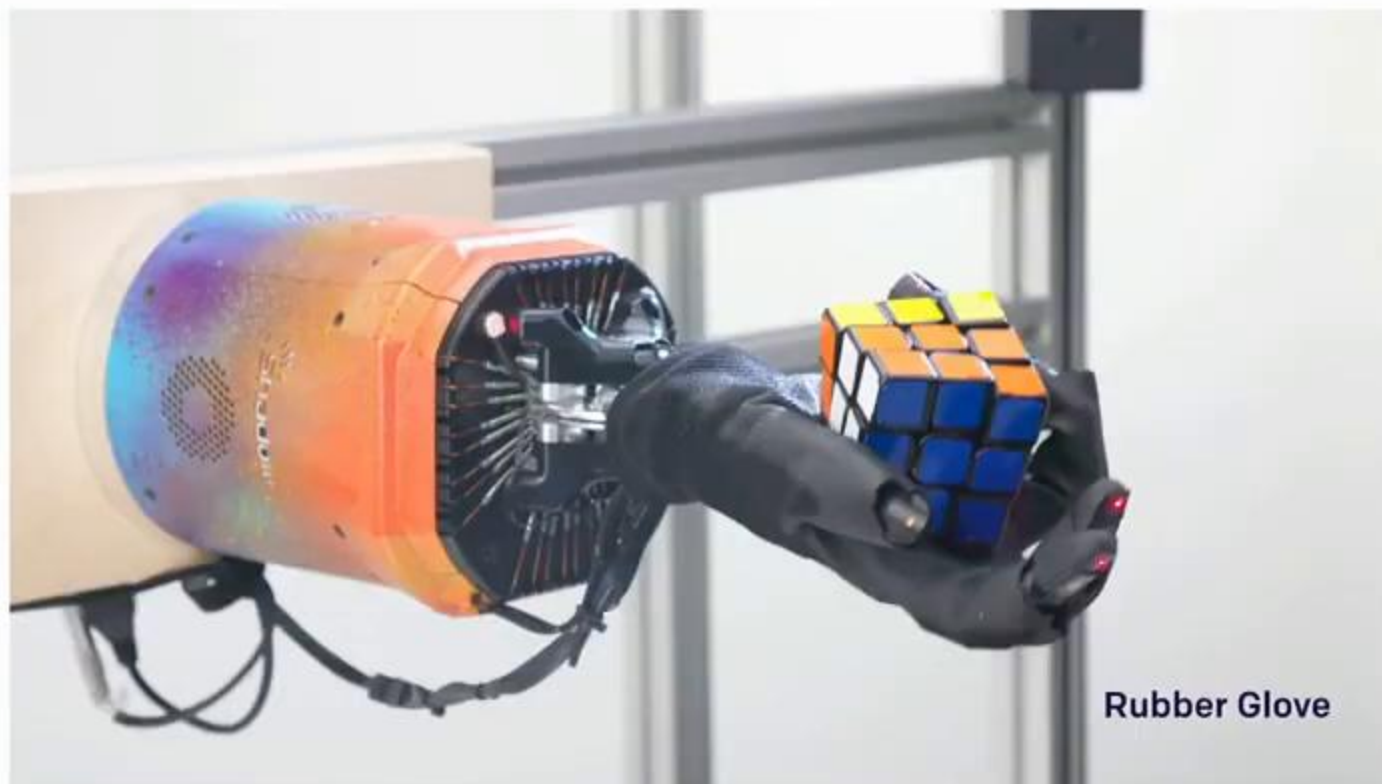
- ✓ Scales linearly $O(N)$ with the number of trainable parameters
- ✓ Significantly easier to implement, works with standard optimizers like Adam
- ✓ Foundation to train LLMs with RLHF or GRPO
- ✗ No monotonic improvement guarantee
- ✗ Near-on-policy, needs many samples

Proximal Policy Optimization Algorithms

Schulman, et al., (2017)

Case Study: OpenAI Dextrous Hand

- PPO sim2real achieves human-level dexterity on 24 DoF



Solving Rubik's Cube With a Robot Hand
OpenAI et al., (2019)

Full PPO Objective

$$L^{PPO}(\theta') = E_{s,a \sim \pi_{\theta}} [\underbrace{L^{CLIP}(\theta')}_{\text{Clipped Surrogate Objective}} - \underbrace{c_1(V_{\theta'}(s) - V^{targ})^2}_{\text{For Advantage}} + \underbrace{c_2 H(\pi_{\theta'})(s)}_{\text{Entropy Bonus}}]$$

Clipped Surrogate
Objective

For Advantage

Entropy Bonus

“Actor”

“Critic”

✓ Combines the best of value-based methods and policy gradients!

Actor Critic Methods

- **Actor:** uses Policy Gradients to predict continuous actions
- **Critic:** uses Value learning to evaluate goodness of a state
- **Actor-Critic:**  Low variance from critic, continuous actions from actor, direct policy optimization
 - Learns from every step via TD updates, at the cost of small bootstrapping bias
 - Optimal baseline to lower variance via advantage
 - Robotics: often asymmetric information via privileged info for critic
 - ✗ Actor and Critic still near-on-policy, needs many samples

Soft Actor-Critic

- Critic trained fully off-policy on a replay buffer!

Exploration: Entropy Bonus

$$L^{Actor}(\theta') = \mathbb{E}_{s \sim \mathbb{D}_{\text{replay}}, a \sim \pi_{\theta'}} [\alpha H(\pi_{\theta'})(s) - Q_{\phi}(s, a)]$$

Exploitation: if I were to do this action today, how many points would I get?

target network

$$L^{Critic}(\phi) = \mathbb{E}_{\tau \sim \mathbb{D}_{\text{replay}}} \left[\left(Q_{\phi}(s, a) - \left(r + \gamma \left(Q_{\bar{\phi}}(s', a') + \alpha H(\pi_{\theta'})(s') \right) \right) \right)^2 \right]$$

actual reward received

How many points did I think this past action a in this past situation s was worth?

this state leads to high reward AND gives you many different ways to succeed

Reparameterization Trick

- Problem: Update Actor based on the Critic's score.

$$a \sim \mathcal{N}(\mu_{\theta'}(s), \sigma_{\theta'}(s)) \rightarrow \text{Stochastic \& Non-differentiable}$$



Mean Std Dev
Action Dist. Action Dist.

$$a = f_{\theta'}(s, \epsilon) = \mu_{\theta'}(s) + \sigma_{\theta'}(s) \odot \epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(0,1)$$


sampled from a fixed distribution without
weights, so no gradients needed!

$$L^{Actor}(\theta') = \mathbb{E}_{s \sim \mathbb{D}_{\text{replay}}, \epsilon \sim \mathcal{N}(0,1)} [\alpha H(\pi_{\theta'})(s) - Q_{\phi}(s, f_{\theta'}(s, \epsilon))]$$

Conclusion

- REINFORCE, optimize policy directly
 - Causality + Baseline: reduce variance
 - Advantage Function: $V(s)$ as optimal baseline
- Importance Sampling: multiple gradient steps on batch
- Entropy Regularization: maintain support
- TRPO: bound how much policy changes via KL
- PPO: Clip ratio instead, still near-on-policy
- SAC: fully off-policy critic via replay buffer, most sample efficient

Thank you for your attention